YouTube video link – https://www.youtube.com/watch?v=rzydN90DxeA

System Name: AshenLens: Spatially-Grounded Multimodal RAG on Ashen Era

Team Name: GAMAGES

Github Repository link - https://github.com/Hasanga910/The-Ashen-Era

1. Problem Statement and Chosen Sub-track

Chosen Track: Sub-track 1A (Rich Answers, Not Just Text).

Problem Statement: Real-world enterprise documentation mixes diagrams, schematics, ledgers, and tables across thousands of heterogeneous pages. Traditional text-only Retrieval-Augmented Generation (RAG) systems fail enterprise users because they merely synthesize prose, forcing technicians and analysts to manually dig through documents to locate and verify critical visual evidence.

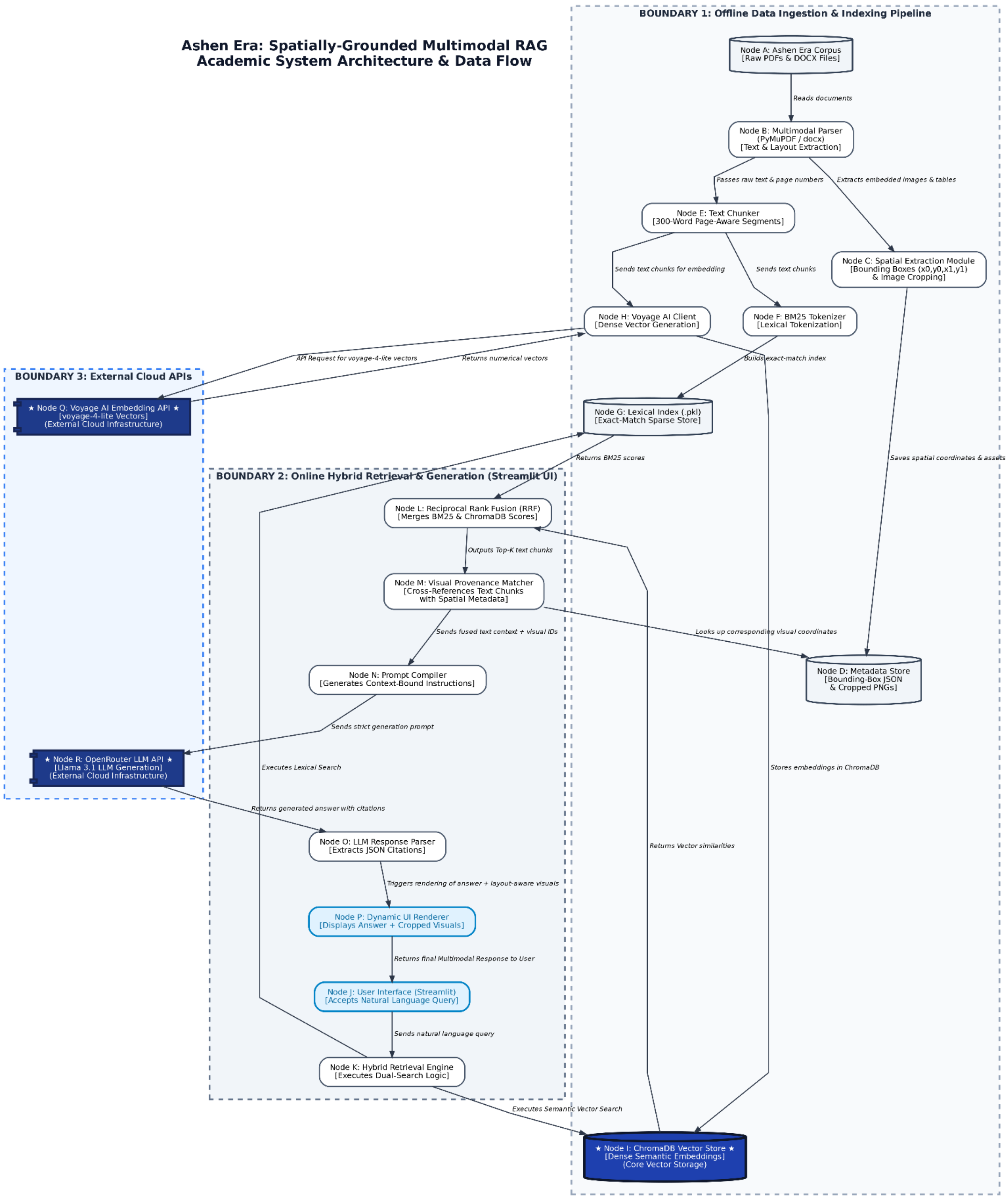
Domain Challenge: The Ashen Era Archive contains 415 documents (1,277 pages) across mixed formats (PDF, DOCX, Markdown, scans). We engineered an assistant to deliver strictly grounded answers alongside dynamically extracted, precise visual evidence.

2. Solution Overview and Architecture

AshenLens is a layout-aware, multimodal RAG pipeline engineered to solve weak visual grounding. Rather than outputting unstructured text or uncurated full-page screenshots, our architecture preserves and tracks exact spatial coordinates (x_0, y_0, x_1, y_1) for every visual artifact.

Core Workflow:

1. **Ingestion:** PyMuPDF extracts text chunks, tables, and images, saving exact (x0, y0, x1, y1) bounding-box coordinates to a visual metadata JSON store.
2. **Indexing & Retrieval:** We employ a Hybrid Search architecture. Text chunks are indexed semantically using Voyage AI embeddings (stored in ChromaDB) and lexically using BM25. User queries are routed through both, and the results are merged using Reciprocal Rank Fusion (RRF).
3. **Visual Provenance Matching:** Retrieved text chunks are cross-referenced with our visual metadata. If the cited page contains a relevant visual asset.
4. **Generation & UI:** OpenRouter (Llama 3.1) synthesizes an answer with strict [Document, Page] citations. The Streamlit interface displays the text, the cropped visual evidence, and offers an "Inspect Source Context" feature that draws a dynamic red bounding box over the full original page render.


Ashen Era: Spatially-Grounded Multimodal RAG
Academic System Architecture & Data Flow
BOUNDARY 1: Offline Data Ingestion & Indexing Pipeline
Node A: Ashen Era Corpus
[Raw PDFs & DOCX Files]
Reads documents
Node B: Multimodal Parser
(PyMuPDF / docx)
[Text & Layout Extraction]
Passes raw text & page numbers
Extracts embedded images & tables
Node E: Text Chunker
[300-Word Page-Aware Segments]
Node C: Spatial Extraction Module
[Bounding Boxes (x0,y0,x1,y1)
& Image Cropping]
Sends text chunks for embedding
Sends text chunks
Node H: Voyage AI Client
[Dense Vector Generation]
Node F: BM25 Tokenizer
[Lexical Tokenization]
API Request for voyage-4-lite vectors
Returns numerical vectors
Builds exact-match index
BOUNDARY 3: External Cloud APIs
★ Node Q: Voyage AI Embedding API ★
[voyage-4-lite Vectors]
(External Cloud Infrastructure)
Node G: Lexical Index (.pkl)
[Exact-Match Sparse Store]
Returns BM25 scores
Saves spatial coordinates & assets
BOUNDARY 2: Online Hybrid Retrieval & Generation (Streamlit UI)
Node L: Reciprocal Rank Fusion (RRF)
[Merges BM25 & ChromaDB Scores]
Outputs Top-K text chunks
Node M: Visual Provenance Matcher
[Cross-References Text Chunks
with Spatial Metadata]
Sends fused text context + visual IDs
Looks up corresponding visual coordinates
Node N: Prompt Compiler
[Generates Context-Bound Instructions]
Node D: Metadata Store
[Bounding-Box JSON
& Cropped PNGs]
Sends strict generation prompt
★ Node R: OpenRouter LLM API ★
[Llama 3.1 LLM Generation]
(External Cloud Infrastructure)
Executes Lexical Search
Stores embeddings in ChromaDB
Returns generated answer with citations
Node O: LLM Response Parser
[Extracts JSON Citations]
Returns Vector similarities
Triggers rendering of answer + layout-aware visuals
Node P: Dynamic UI Renderer
[Displays Answer + Cropped Visuals]
Returns final Multimodal Response to User
Node J: User Interface (Streamlit)
[Accepts Natural Language Query]
Sends natural language query
Node K: Hybrid Retrieval Engine
[Executes Dual-Search Logic]
Executes Semantic Vector Search
★ Node I: ChromaDB Vector Store ★
[Dense Semantic Embeddings]
(Core Vector Storage)

3. Key Technical Decisions

- **Spatial Bounding-Box Extraction vs. Page Screenshots:** We decided against displaying generic, full-page screenshots as visual evidence. Full pages overwhelm users and look cluttered on web interfaces. By engineering our pipeline to extract spatial bounding boxes.
- **Hybrid Search with Reciprocal Rank Fusion (RRF):** Early testing revealed that pure dense vector search struggled with exact, out-of-vocabulary fantasy names. We integrated a BM25 lexical index to capture exact keyword matches, fusing the scores with our ChromaDB semantic vectors to guarantee high recall for both lore concepts and specific entities.
- **Cloud LLM over Local Inference:** We initially explored using Ollama to run models locally and avoid API rate limits. However, we pivoted to the OpenRouter API. This ensured lightning-fast generation speeds during our live demonstration and prevented hardware bottlenecks on standard student laptops.

4. What Works, What Doesn't, and Limitations

What Works Flawlessly:

- Text extraction, chunking, and strict citation generation from standard PDFs and DOCX files.
- Hybrid retrieval successfully bridges the gap between semantic meaning and exact fantasy terminology.
- Dynamic visual cropping and UI bounding-box overlays render perfectly for embedded images and tables.

Failed Approaches & Limitations:

- **Failure - Tesseract OCR on Simulated Scans:** We originally built an ocr_ingest.py script utilizing Tesseract to read the simulated 18th-century ephemera. During testing, the stylized fantasy fonts produced highly noisy, corrupted text. This noise actively degraded the precision of our BM25 index.
- **Strategic Limitation:** Following this failure, we made the deliberate engineering choice to deprecate the OCR pipeline. As a result, purely image-based scanned ephemera lacking a hidden text layer cannot currently be searched lexically. We accepted this trade-off to prioritize zero-hallucination accuracy and high-fidelity visual grounding on the primary canonical texts (Codices, Novels, Wiki).

5. AI Usage Disclosure – Refer the README file for complete AI usage chatlogs

In accordance with the competition's AI Usage Policy, we disclose the following:

- **Tools Used:** Claude 3.5 Sonnet, ChatGPT (GPT-4o), Gemini Pro
- **Purpose:** Used as pair-programming assistants to accelerate the development of Streamlit UI components, refine strict JSON citation prompts for the OpenRouter API
- **Human-Led Decisions:** All strategic architectural choices -including the transition from pure vector search to Hybrid RRF, the bounding-box coordinate math, and the decision to abandon OCR were strictly team-driven. We actively overrode AI suggestions that recommended unstable libraries, ensuring every line of committed code was fully understood and locally verifiable by the team.

6. Team Member Roles and Contributions

All repository-level integration, environment standardization (requirements.txt), documentation, and rigorous QA testing were shared collaboratively across the team. Core module ownership was distributed as follows:

- **Dunith Desitha Ranawansha - Document Ingestion Lead**
  - **Owned:** src/ingest/ (pdf_ingest.py, docx_ingest.py, and the deprecated ocr_ingest.py).
  - **Contributions:** Engineered the raw document processing pipeline to handle the enterprise corpus. Implemented the layout-aware extraction logic to capture text alongside precise spatial bounding-box coordinates for images and tables.
- **Tharidi Pabasari Gamage - Processing & Indexing Lead**
  - **Owned:** src/indexing/ (chunker.py, embeddings.py, vector_index.py, build_all.py).
  - **Contributions:** Architected the transformation of extracted raw content into searchable representations. Designed the chunking strategy and successfully mapped the embeddings pipeline directly into the ChromaDB vector store.
- **Sajana Hasanga - Retrieval Lead**
  - **Owned:** src/retrieval/ (hybrid_search.py).
  - **Contributions:** Engineered the core search engine responsible for finding the most relevant evidence. Successfully implemented the BM25 + dense vector retrieval integration, Reciprocal Rank Fusion (RRF), and the visual-intent prioritization logic.

- **Yasindu Sasmitha - Generation & Application Lead**
  - **Owned:** src/generation/ (answer.py) and src/app.py.
  - **Contributions:** Transformed retrieved evidence into the final cited answer and built the interactive Streamlit UI. Integrated Qwen vision handling, dynamic visual evidence selection, and the deterministic citation fallback mechanisms to guarantee zero-hallucination outputs.